Roteiro de Estudo e Diário de Bordo — Revisão

Oficina presencial de revisão para a prova e preparação da apresentação · Modelagem de Dados e Banco de Dados Relacional

Grupo	7 — ERP Petly Club
Turma	_____
Data	___/___/2026 — preenchido em 30/10/2026
Integrantes presentes	_____

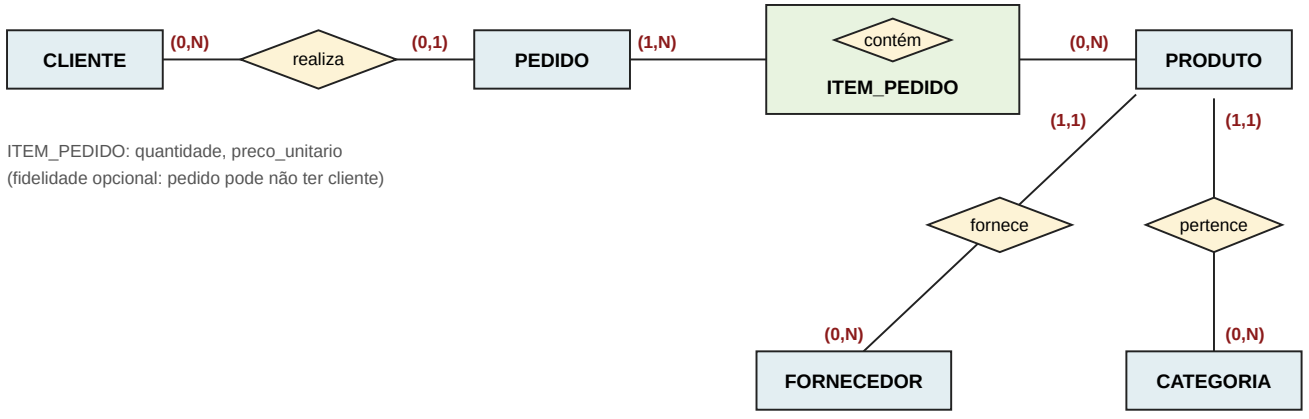
> **Caso de referência:** Cafeteria & Grãos Artesanais — Sabor & Ponto. Atividade em papel, sem SGBD. As respostas dos blocos 1 a 4 foram conferidas depois com o mesmo caso implementado no PostgreSQL no Laboratório Prático Integrado.

Bloco 1 — Negócio, regras e DER

Quatro entidades são suficientes para representar todas as regras?

Não. As quatro principais são Cliente, Pedido, Produto e Categoria, mas: (1) o **fornecedor** tem atributos próprios (nome e CNPJ) e é compartilhado por vários produtos — se ficasse dentro de Produto, o nome do fornecedor se repetiria em cada produto (problema de 3FN); (2) Pedido e Produto têm relação **N:N**, e a quantidade vendida não pertence a nenhum dos dois — é preciso a entidade associativa **ItemPedido**. Total: 6 entidades.

DER com cardinalidades mínimas e máximas



ITEM_PEDIDO: quantidade, preco_unitario
(fidelidade opcional: pedido pode não ter cliente)

Cliente sem pedido: o mínimo do lado do cliente é 0 — CLIENTE (0,N), pode estar cadastrado e ainda não ter comprado. **Pedido com itens obrigatórios:** o mínimo do pedido em ItemPedido é 1 — PEDIDO (1,N), não existe pedido vazio. Como o cadastro de fidelidade é opcional, o pedido participa com (0,1) do lado do cliente.

Por que ItemPedido é necessário e quais atributos pertencem a ele?

Porque um pedido tem vários produtos e um produto aparece em vários pedidos (N:N), e o modelo relacional não representa N:N diretamente. ItemPedido guarda o que pertence à

combinação pedido + produto: **quantidade** e **preço unitário praticado** (o preço do catálogo pode mudar; o da venda não).

Duas regras de negócio que devem virar restrições

1) “A quantidade vendida e o preço são positivos” → `CHECK (quantidade > 0)` e `CHECK (preco > 0)`. 2) “O mesmo CPF não pode ter dois cadastros de fidelidade” → `UNIQUE (cpf)`. Também: “todo produto pertence a uma categoria” → FK `id_categoria NOT NULL`.

Bloco 2 — Modelo lógico e normalização

Tabelas com PK e FK

```
CATEGORIA   (id_categoria PK, nome)
FORNECEDOR  (id_fornecedor PK, nome, cnpj)
PRODUTO     (id_produto PK, nome, preco, id_categoria FK, id_fornecedor FK)
CLIENTE     (id_cliente PK, nome, cpf, telefone, email)
PEDIDO      (id_pedido PK, data_pedido, id_cliente FK – aceita nulo)
ITEM_PEDIDO (id_pedido PK/FK, id_produto PK/FK, quantidade, preco_unitario)
```

Chave composta (id_pedido, id_produto) ou identificador próprio em ItemPedido?

Chave composta: já identifica o item de forma única e ainda impede, pelo próprio banco, o mesmo produto em duas linhas do mesmo pedido. Com um **id próprio** (id_item), essa regra se perderia, a menos que se acrescentasse `UNIQUE (id_pedido, id_produto)`. O id próprio só valeria a pena se outra tabela precisasse referenciar o item (ex.: devolução de um item). No projeto Petly Club, a chave composta é referenciada pela movimentação de estoque por uma FK composta.

Problemas de Pedido(id_pedido, data, nome_cliente, telefone_cliente, produto1, produto2)

1FN: produto1 e produto2 são um **grupo repetitivo** — limitam o pedido a 2 produtos, deixam colunas vazias e não têm onde guardar a quantidade de cada um. Correção: tabela ItemPedido, uma linha por produto. **Redundância:** nome e telefone do cliente se repetem em todo pedido; se o telefone mudar, é preciso atualizar vários pedidos, e um pode ficar diferente do outro (anomalia de atualização). Além disso, nome_cliente depende do cliente, e não do pedido (dependência transitiva). Correção: tabela Cliente e só a FK id_cliente em Pedido.

Como verificar 2FN e 3FN

2FN (só faz diferença com chave composta): todo atributo não chave depende da chave **inteira**? Em ItemPedido(id_pedido, id_produto, quantidade, nome_produto): id_produto → nome_produto depende só de parte da chave — dependência parcial; nome_produto vai para Produto. **3FN:** algum atributo não chave depende de **outro atributo não chave**? Em Produto(id_produto, nome, id_fornecedor, nome_fornecedor): id_produto → id_fornecedor → nome_fornecedor é transitiva; nome_fornecedor vai para Fornecedor. Atributos derivados, como valor_total do pedido, também saem: dependem dos itens.

Bloco 3 — SQL em papel: DDL, DML e integridade

```
CREATE TABLE cliente (
    id_cliente INTEGER PRIMARY KEY,
    nome       VARCHAR(120) NOT NULL,
    cpf        CHAR(11)     NOT NULL
);
CREATE TABLE pedido (
    id_pedido   INTEGER PRIMARY KEY,
    data_pedido TIMESTAMP NOT NULL,
    id_cliente  INTEGER REFERENCES cliente (id_cliente)       -- fidelidade opcional
);
CREATE TABLE item_pedido (
    id_pedido      INTEGER       NOT NULL REFERENCES pedido (id_pedido),
    id_produto     INTEGER       NOT NULL REFERENCES produto (id_produto),
    quantidade     INTEGER       NOT NULL CHECK (quantidade > 0),
    preco_unitario NUMERIC(10,2) NOT NULL CHECK (preco_unitario > 0),
    PRIMARY KEY (id_pedido, id_produto)
);
```

CHECK: impede registrar quantidade 0 ou negativa e preço zero ou negativo — por exemplo, um erro de digitação no caixa que geraria venda “negativa” e bagunçaria o caixa e o estoque. O banco responde com `violates check constraint`.

```
-- válido: o pedido 1 e o produto 3 existem
INSERT INTO item_pedido VALUES (1, 3, 2, 9.50);
-- viola FK: não existe produto 99
INSERT INTO item_pedido VALUES (1, 99, 1, 10.00);
-- mensagem esperada: insert or update on table "item_pedido" violates foreign key constraint
...
--                    Key (id_produto)=(99) is not present in table "produto".
```

```
UPDATE produto SET preco = 12.00 WHERE id_produto = 5;
DELETE FROM item_pedido WHERE id_pedido = 4 AND id_produto = 4;
```

Sem WHERE, o UPDATE mudaria o preço de **todos** os produtos para R$ 12,00, e o DELETE apagaria **todos** os itens de todos os pedidos. O banco aceitaria, porque nenhuma restrição é violada — o WHERE é a única proteção.

```
ALTER TABLE cliente ADD CONSTRAINT uk_cliente_cpf UNIQUE (cpf);
```

Se já houver CPFs repetidos, o ALTER é recusado. Antes, é preciso encontrá-los — `SELECT cpf, COUNT(*) FROM cliente GROUP BY cpf HAVING COUNT(*) > 1;` —, decidir qual cadastro fica, apontar os pedidos do cadastro duplicado para o que fica (`UPDATE pedido SET id_cliente = ...`), excluir o duplicado e só então criar a restrição.

Bloco 4 — Consultas e interpretação

```
SELECT nome, preco
FROM produto
WHERE preco > 15.00
ORDER BY preco DESC;

SELECT p.nome AS produto, c.nome AS categoria
FROM produto p
INNER JOIN categoria c ON c.id_categoria = p.id_categoria;

SELECT pe.id_pedido, pe.data_pedido, c.nome AS cliente,
       SUM(i.quantidade * i.preco_unitario) AS valor_total
FROM pedido pe
JOIN cliente c     ON c.id_cliente = pe.id_cliente
JOIN item_pedido i ON i.id_pedido  = pe.id_pedido
GROUP BY pe.id_pedido, pe.data_pedido, c.nome;
```

Por que juntar ItemPedido e Produto exige conhecer as FKs: a junção tem de usar a coluna que realmente liga as tabelas (`item_pedido.id_produto = produto.id_produto`). Juntar pela coluna errada (por exemplo, id_pedido = id_produto) roda sem erro, mas mistura produtos com pedidos que não têm nada a ver — o resultado sai errado em silêncio. Em ItemPedido a chave é composta, e cada parte é FK para uma tabela diferente.

Troca de folhas — acerto, erro e melhoria

Consulta	Acerto	Erro	Melhoria
Produtos > R$ 15	Filtro e ordenação corretos	Usar >= incluiria produtos de exatamente R$ 15,00, o que o enunciado não pede	Mostrar também a categoria, para a gerência
Produto × categoria	JOIN pela FK correta	—	Ordenar por categoria e produto
Pedido × cliente	Total calculado dos itens, sem coluna valor_total	Com `JOIN` (inner), vendas sem cadastro de fidelidade não aparecem	`LEFT JOIN cliente` para listar todas as vendas

5. Preparação da apresentação — simulação da banca

Pontos-chave que o grupo usou como referência (projeto Petly Club). Cada integrante responde oralmente, sem ler.

Pergunta	Pontos-chave da resposta
1. Qual problema real o ERP resolve e quem se beneficia?	Lojas física e virtual com estoque único, mas sistemas e planilhas que não se comunicam: cadastros duplicados, estoque desatualizado, retrabalho no financeiro. Beneficia gerência, caixa, estoque, financeiro e o cliente do clube.
2. Por que essas entidades e relacionamentos?	Cada uma vem de uma regra numerada; LOTE e MOVIMENTACAO_ESTOQUE para validade e histórico; CONTA_RECEBER para separar autorizado de recebido.

3. Uma relação N:N e como foi resolvida	Pedido × Produto → ITEM_PEDIDO, com quantidade e preço praticado, chave composta.
4. Onde estão PK e FK; que integridade garantem?	16 PK e 21 FK, todas com RESTRICT; FK composta (lote, produto) impede vender de lote de outro produto.
5. Uma decisão de normalização até a 3FN	Saldos do lote e total do pedido deixaram de ser colunas (derivados) e viraram visões.
6. Uma restrição CHECK, UNIQUE ou NOT NULL	`ck_pedido_canal_recebimento`: loja física só vende no balcão; `uk_cliente_cpf`: CPF não repete.
7. Uma consulta e sua utilidade gerencial	Produtos no estoque mínimo ou abaixo → gerar a solicitação de compra.
8. Limitação ou melhoria futura	Parte das regras (escolha FEFO do lote) fica na aplicação; próximo passo: dashboard ligado ao banco e previsão de demanda com dados reais.

Registro da simulação

Integrante	Pergunta	Respondeu com clareza?	O que revisar

6. Diário de bordo do grupo

O que revisamos hoje? (conceitos e exemplos)

Entidades, cardinalidades mínimas e máximas e entidade associativa (ItemPedido); conversão do DER em tabelas com PK e FK; chave composta × identificador próprio; 1FN, 2FN e 3FN com dependências funcionais; DDL com CHECK, UNIQUE e FK; INSERT válido e inválido; UPDATE e DELETE com WHERE; ALTER TABLE; SELECT, INNER JOIN e LEFT JOIN — tudo no caso da cafeteria e comparado com o projeto Petly Club.

Quais erros identificamos e corrigimos?

No caso: consulta de pedidos com INNER JOIN omitia vendas sem cliente de fidelidade (corrigido com LEFT JOIN). No projeto: o DER não mostrava os relacionamentos de autorização [A2] e [A3] que já estavam no modelo lógico; o README ainda era o da primeira entrega; o registro de prompts citava “PEPS”, enquanto a regra do projeto e o script usam FEFO. Os três foram corrigidos no repositório.

Que dúvidas permanecem para a prova?

Diferença entre entidade fraca e entidade associativa em casos de exemplo; quando escolher chave composta ou identificador próprio; ações de FK (RESTRICT, CASCADE, SET NULL).

Que ajustes são necessários antes da apresentação?

Montar os slides e o roteiro de 20 minutos (explicar, mostrar, defender, demonstrar, propor); escolher a demonstração ao vivo (gatilho recusando venda sem saldo e uma consulta gerencial); ensaiar as respostas da arguição; gerar o PDF da documentação para avaliação às cegas.

Que evidência de aprendizagem cada integrante apresentou?

Quais são os responsáveis e prazos para os próximos ajustes?

7. Autoavaliação individual

Nome: _____ RGM: _____

☐ Seguro ☐ Parcial ☐ Preciso estudar — Consigo identificar entidades, atributos e cardinalidades.
☐ Seguro ☐ Parcial ☐ Preciso estudar — Consigo converter DER em tabelas com PK e FK.
☐ Seguro ☐ Parcial ☐ Preciso estudar — Consigo justificar a normalização até a 3FN.
☐ Seguro ☐ Parcial ☐ Preciso estudar — Consigo escrever e interpretar DDL e DML.
☐ Seguro ☐ Parcial ☐ Preciso estudar — Consigo explicar SELECT e JOIN sem executar o código.
☐ Seguro ☐ Parcial ☐ Preciso estudar — Consigo defender tecnicamente uma decisão do projeto.

Minha prioridade de estudo para a prova: _____

Minha contribuição concreta para a apresentação: _____

8. Depois da aula — missão de consolidação

☐ Refazer, individualmente, duas questões em que tive dificuldade.
☐ Conferir a coerência entre DER, modelo lógico, dicionário de dados e SQL do projeto.
☐ Preparar uma explicação oral de 90 segundos sobre uma decisão técnica do grupo.
☐ Revisar as consultas SQL e testar mentalmente os resultados esperados.
☐ Registrar no grupo as correções e responsabilidades combinadas.